A scheduling approach with GNN and reinforcement learning for DAG workflows in homogeneous cloud environments

Haotian Duan[1], Zijun Luo[1], Chenyu Feng[1], Lingjuan Ye[2]

1. Queen Mary School Hainan, Beijing University of Posts and Telecommunications, Beijing, China

2. School of Mathematical Sciences, Beijing University of Posts and Telecommunications, Beijing, China
E-mail: ecurve@bupt.edu.cn, becky.luo@outlook.com, 1844936768@qq.com, lingjuanye@126.com

Abstract: We study pipeline-style workflow scheduling where a workflow is modeled as a DAG and tasks generate downstream work during execution. A fixed pool of identical executors must be allocated to concurrently ready work items, and the goal is to minimize the makespan under precedence constraints and resource contention. We propose two complementary schedulers: (i) a pipeline-adapted DBL heuristic that extends dynamic priority ideas to the pipeline propagation model, and (ii) a learning-based scheduler that combines graph neural networks with reinforcement learning (GNN+RL) to learn allocation policies from DAG states. We evaluate against rank-based scheduling and ACO on a synthetic benchmark spanning $N = 100$–1000 nodes and average executor regimes $E = 2$–8. GNN+RL gives the strongest overall quality and a favorable quality–runtime trade-off compared with meta-heuristic search.

Key Words: DAG workflow scheduling; graph neural network; reinforcement learning; homogeneous cloud environments

1 Introduction

Pipeline-style workflow scheduling arises in many data-intensive and scientific applications where tasks form a directed acyclic graph (DAG) and must be executed under limited computing resources. Recent workflow benchmarks and learning-based schedulers further show that reproducible DAG generation and structure-aware policies are central to scalable scheduling studies.[1, 3, 7, 8] The key objective is to minimize the overall completion time (makespan) while respecting precedence constraints and resource capacity.

Scheduling DAG workflows is challenging for three main reasons: (i) precedence constraints create complex critical-path effects; (ii) limited executors force contention among concurrently ready tasks; and (iii) resource allocation decisions are highly coupled over time, where suboptimal early choices can propagate delays downstream. The resulting optimization problem is NP-hard in general, and exact methods are often infeasible at the scale and variability seen in real pipelines.

This paper studies pipeline scheduling under a realistic execution model where tasks generate work dynamically along DAG edges and resources are homogeneous but limited. We develop and evaluate heuristic and learning-based schedulers that are lightweight enough for practical deployment and robust across varying DAG sizes and executor regimes.

Our main contributions are:

- **Pipeline-adapted DBL.** We extend the DBL heuristic to the pipeline execution model by designing a priority mechanism that remains effective under dynamic work propagation and executor contention.
- **GNN+RL scheduler.** We propose a learning-based scheduler that combines graph neural networks (GNNs) with reinforcement learning (RL) to directly learn allocation policies from DAG states, enabling generalization across unseen workflow structures and scales.
- **Empirical evaluation.** We provide systematic simulation results on synthetic DAG benchmarks spanning different graph sizes and executor regimes, demonstrating improvements in makespan and favorable quality–runtime trade-offs.

This research was supported by by the National Natural Science Foundation of China (62303066) (Corresponding author: Lingjuan Ye, Email: lingjuanye@126.com)

2 Problem Description

2.1 Pipeline Scheduling Model

We model a workflow as a directed acyclic graph (DAG) $G = (\mathcal{T}, \mathcal{E})$, where each node $u \in \mathcal{T}$ is a pipeline stage and each directed edge $(u \to v) \in \mathcal{E}$ indicates that stage u precedes stage v.

Node (task) attributes. Each node $u \in \mathcal{T}$ is associated with:

- a required *work count* $w_u \in \mathbb{Z}_{\geq 0}$ (the number of work items that must be processed at u), and
- a *per-item processing time* $\tau_u > 0$, so that completing one work item at u consumes τ_u time on one executor.

Thus, if node u processes k work items, its total processing time on a single executor is $k\tau_u$.

Executor attributes. We are given R homogeneous executors. Each executor can process at most one work item at any time. Executors are *non-preemptive* and *non-migratory*: once an executor starts a work item, it runs to completion on that executor, and the assignment cannot be changed mid-execution.

Work propagation. Pipeline semantics are modeled as dynamic work generation along edges. When node u completes one work item, it immediately triggers work for all its children: for every edge $(u \to v)$, the outstanding work count of v increases by one. Equivalently, each completed item at u produces one item for each successor v:

$$A_v(t^+) \leftarrow A_v(t^-) + 1 \quad \forall (u \to v) \in \mathcal{E}, \tag{1}$$

where $A_v(t)$ denotes the number of available work items at node v. At time $t = 0$, all executors are available and execution starts simultaneously. Initially, source nodes (in-degree

0) have their given w_u available ($A_u(0) = w_u$), while other nodes may receive work only after predecessors finish items.

Objective. A scheduling policy determines, at each decision point, which nodes' available work items are assigned to the R executors. Let $C_{\max}$ denote the completion time of the last finished work item in the entire workflow (the makespan). The goal is:

$$\min_{\pi} \; C_{\max}(\pi) \tag{2}$$

subject to the resource and feasibility constraints

$$\begin{aligned} &\sum_{u \in \mathcal{T}} y_{r,u}(t) \le 1, \\ &\sum_{r=1}^{R} y_{r,u}(t) \le A_u(t), \\ &\qquad y_{r,u}(t) = 0 \quad \text{if } A_u(t) = 0, \end{aligned} \tag{3}$$

where $y_{r,u}(t) \in \{0, 1\}$ indicates whether executor r processes a work item of node u at time t. These constraints enforce one task per executor, availability of work, and precedence through the propagation rule above.

2.2 Problem Analysis

This pipeline model introduces strong temporal coupling and nontrivial queueing effects. First, work availability is *dynamic* (generated by upstream completions), so the set of runnable items changes continuously. Second, assigning executors to one node can starve another node that lies on the critical path, causing amplified downstream delays. Third, even with homogeneous executors, the best decision depends on both local node attributes (w_u, τ_u) and global DAG structure (fan-out, depth, and remaining downstream work). These factors yield a large, nonconvex decision space and make optimal scheduling computationally intractable at scale. Therefore, exact optimization is avoided; we compare classical rank-based allocation, DBL, ACO, and the proposed GNN+RL policy, with the last method as the main contribution.

3 Proposed Algorithms

We present four pipeline resource allocation algorithms for a workflow represented as a directed acyclic graph (DAG), namely Upward Rank, DBL, ACO, and GNN, together with a simulation-based evaluator used to compute the final performance metric (makespan)[1, 7]. The first three methods are adapted baselines: rank and DBL rules use fixed critical-path or level proxies, while recent ACO-style schedulers search iteratively and can be costly.[6] The main proposed method is GNN+RL, which learns the proxy from graph states.

3.1 Notation Used by the Algorithms

Table 1 summarizes the symbols used by the constructive allocation algorithms. The vector $\mathbf{e} = [e_1, \ldots, e_{|\mathcal{T}|}]^\top$ denotes the current integer executor allocation, and each completed allocation is evaluated by the discrete-event simulator.

Table 1: Important notation

Symbol	Meaning
$G = (\mathcal{T}, \mathcal{E})$	DAG workflow (nodes/tasks and edges /dependencies)
$\lvert\mathcal{T}\rvert, \lvert\mathcal{E}\rvert$	Number of tasks and edges
e_i	Executors allocated to task (node) t_i ($e_i \in \mathbb{Z}_{\ge 0}$)
E_{tot}	Total executor budget ($\sum_i e_i = E_{\text{tot}}$)
$C_{\max}(\mathbf{e})$	Makespan under allocation $\mathbf{e}$ (returned by simulator)
$A_i(t), y_{r,i}(t)$	Available work and executor-task indicator
$\mathrm{succ}(t_i)$	Successor set of t_i in G
$\mathrm{pred}(t_i)$	Predecessor set of t_i in G
$\widehat{p}_i(e_i)$	Surrogate processing-time model for t_i under e_i executors
$\mathrm{rank}u(t_i)$	Upward rank (critical-path-to-exit surrogate)
ℓ_i	DBL level index (bottom-up level)
τ_i, η_i	ACO pheromone / heuristic value for selecting t_i
α, β, ρ, Q	ACO parameters (weights, evaporation, deposit scale)
$\pi_\theta(\cdot\lvert s)$	GNN policy (actor) with parameters θ
$V_\phi(s)$	Value function (critic) with parameters ϕ

3.2 Upward Rank Based Greedy Allocation

Recent workflow schedulers still use upward-rank or HEFT-style priorities as low-cost critical-path proxies for DAG allocation.[2, 3, 5]

To adapt upward rank to *resource allocation* (instead of machine assignment), we introduce a lightweight surrogate processing-time model:

$$\widehat{p}_i(e_i) = \begin{cases} 0, & w_i = 0, \\ \left\lceil \dfrac{w_i}{e_i} \right\rceil \cdot \bar{d}_i, & w_i > 0,\ e_i \ge 1, \\ +\infty, & w_i > 0,\ e_i = 0. \end{cases} \tag{4}$$

where w_i is the number of work units (or aggregated items) handled by node t_i, and $\bar{d}_i$ is the average time per work unit.

We define an upward-rank recursion in this rank-priority style while omitting communication terms (or absorbing them into a deterministic edge cost $\kappa_{i,j}$ when needed):

$$\mathrm{rank}_u(t_i) = \widehat{p}_i(e_i) + \begin{cases} 0, & \mathrm{succ}(t_i) = \emptyset, \\ \max\limits_{t_j \in \mathrm{succ}(t_i)} \left(\kappa_{i,j} + \mathrm{rank}_u(t_j)\right), & \text{otherwise.} \end{cases} \tag{5}$$

This captures a compute-cost term plus the largest downstream successor cost.

Given the current allocation $\mathbf{e}$, we trace a *surrogate critical path* by repeatedly following the successor that attains the $\max(\cdot)$ in (5). We then allocate each remaining executor to the task on the traced path with the largest marginal reduction:

$$\Delta_i := \widehat{p}_i(e_i) - \widehat{p}_i(e_i + 1). \tag{6}$$

Algorithm 1 Upward Rank Allocation

Input: DAG $G = (\mathcal{T}, \mathcal{E})$, surrogate cost $\widehat{p}_i(\cdot)$, budget E_{tot}
Output: Integer allocation $\mathbf{e}$ with $\sum_i e_i = E_{\text{tot}}$

```
Initialize feasible e; R ← E_tot − Σ_i e_i
while R > 0 do
    Compute rank_u(·) via (5) and extract surrogate critical path P
    i* ← arg max_{t_i ∈ P} Δ_i                ▷ best marginal gain
    e_{i*} ← e_{i*} + 1; R ← R − 1
end while
return e
```

3.3 DBL Leveling with Lower-Bound Guided Allocation

Level-based and deadline-aware workflow schedulers remain common in recent cloud workflow studies because they provide compact precedence summaries for resource allocation.[4, 5]

We compute a DBL-style level index ℓ_i using a bottom-up recursion:

$$\ell_i = \begin{cases} 1, & \text{succ}(t_i) = \emptyset, \\ 1 + \max\limits_{t_j \in \text{succ}(t_i)} \ell_j, & \text{otherwise.} \end{cases} \tag{7}$$

To drive allocation by a bottleneck indicator, we define an abstract flow/workload accumulation:

$$f_i = w_i + \sum_{t_p \in \text{pred}(t_i)} f_p, \tag{8}$$

and an earliest-finish-time style lower bound $\widehat{\text{EFT}}_i$:

$$\widehat{\text{EFT}}_i = \max\left\{ \left\lceil \frac{f_i}{e_i} \right\rceil \bar{d}_i, \max_{t_p \in \text{pred}(t_i)} \widehat{\text{EFT}}_p + \bar{d}_i \right\}. \tag{9}$$

with the convention that the predecessor maximum is 0 for sources.

At each iteration, allocate one additional executor to the task with the largest $\widehat{\text{EFT}}_i$; use ℓ_i as a stable tie-breaker (e.g., prioritize deeper/shallower levels depending on design).

Algorithm 2 DBL-EFT Greedy Allocation

Input: DAG $G = (\mathcal{T}, \mathcal{E})$, workload $\{w_i\}$, unit-time $\{\bar{d}_i\}$, budget E_{tot}
Output: Integer allocation $\mathbf{e}$ with $\sum_i e_i = E_{\text{tot}}$

```
Topologically sort tasks
Compute DBL levels ℓ_i via (7) and flows f_i via (8)
Initialize feasible e; R ← E_tot − Σ_i e_i
while R > 0 do
    Compute EFT̂_i for all tasks via (9)
    i* ← arg max_i EFT̂_i                      ▷ tie-break by larger ℓ_i
    e_{i*} ← e_{i*} + 1;  R ← R − 1
end while
return e
```

3.4 ACO-Based Allocation Search

Recent cloud scheduling studies continue to use ACO variants because pheromone trails and heuristic information provide a simple constructive search mechanism for makespan- and cost-oriented allocation.[6]

We represent an allocation as a sequence of R unit increments (constructive encoding). Let $m_i^{\min}$ be a feasibility lower bound (e.g., $m_i^{\min} = 1$ for active tasks and 0 otherwise), and define

$$R = E_{\text{tot}} - \sum_i m_i^{\min}, \qquad e_i = m_i^{\min} + \sum_{k=1}^{R} \mathbb{I}[a_k = i], \tag{10}$$

where $a_k \in \{1, \ldots, |\mathcal{T}|\}$ is the task chosen at construction step k, and $\mathbb{I}[\cdot]$ is the indicator function.

At each construction step, an ant selects a task according to a pheromone–heuristic product rule:

$$\Pr(a_k = i) = \frac{\tau_i^{\alpha} \eta_i^{\beta}}{\sum_j \tau_j^{\alpha} \eta_j^{\beta}}, \tag{11}$$

where τ_i is the pheromone on task t_i, η_i is a heuristic score (e.g., a marginal-gain proxy), and $\alpha, \beta \geq 0$ control their influence. This form is consistent with ACS-style probabilistic selection.

A completed allocation $\mathbf{e}$ is evaluated by simulation to obtain $C_{\max}(\mathbf{e})$. Pheromone is then updated by evaporation plus elite reinforcement:

$$\tau_i \leftarrow (1 - \rho)\tau_i + \sum_{\mathbf{e} \in \mathcal{S}_{\text{elite}}} \frac{Q}{C_{\max}(\mathbf{e})} g_i(\mathbf{e}), \tag{12}$$

where $\rho \in (0, 1]$ is the evaporation rate, $Q > 0$ is the deposition scale, $\mathcal{S}_{\text{elite}}$ is an elite set (e.g., best-of-iteration and/or global best), and

$$g_i(\mathbf{e}) \;=\; e_i - m_i^{\min} \tag{13}$$

counts how many unit increments were assigned to task t_i beyond the feasibility lower bound.

Algorithm 3 ACO-Based Allocation Search (Constructive Encoding)

Input: DAG G, budget E_{tot}, parameters $(K, I, \alpha, \beta, \rho, Q)$
Output: Best allocation $\mathbf{e}^{\text{best}}$

```
Set lower bounds {m_i^min}; R ← E_tot − Σ_i m_i^min
Initialize pheromones τ_i ← τ_0 and heuristics η_i
e^best ← NULL; C^best ← +∞
for iter = 1 to I do
    S_iter ← ∅
    for ant = 1 to K do
        e_i ← m_i^min for all i
        for k = 1 to R do
            Sample i ~ Pr(a_k = i) using (11)
            e_i ← e_i + 1
        end for
        C ← SimulateMakespan(G, e)
        Add (e, C) to S_iter
        if C < C^best then
            e^best ← e; C^best ← C
        end if
    end for
    S_elite ← EliteSet(S_iter, e^best)
    Update pheromones using (12)
end for
return e^best
```

3.5 GNN-Based Allocation via Reinforcement Learning

Graph neural networks (GNNs) can encode DAG structure via message passing, and reinforcement learning (RL) can learn sequential decision policies for scheduling/resource management from objective-driven feedback, as shown by recent DAG and workflow schedulers using GCN/DRL and GNN-enhanced policies.[3, 7, 8]

3.5.1 MDP Formulation

We formulate incremental allocation as a finite-horizon Markov decision process:

- **State:** $s_k = (G, \mathbf{x}, \mathbf{e}^{(k)})$, where $\mathbf{x}$ contains workload, processing time, in/out-degree, level/rank, source/sink flags, and current allocation.
- **Action:** $a_k \in \mathcal{T}$ selects one task whose executor count is increased by one:

$$e_{a_k}^{(k+1)} = e_{a_k}^{(k)} + 1.$$

- **Horizon:** $R = E_{\text{tot}} - \sum_i e_i^{(0)}$ steps, starting from a feasible base allocation $\mathbf{e}^{(0)}$.
- **Terminal reward:** $r = -C_{\max}(\mathbf{e}^{(R)})$ computed by simulation.

This sequential construction view aligns with RL-based scheduling approaches that incrementally build a scheduling/resource decision.

3.5.2 Graph Encoder and Policy

We employ a message passing neural network (MPNN) as the encoder, following the recent practice of using graph encoders for DAG scheduling.[3, 8] Messages are passed on both predecessor and successor directions so that each node embedding reflects upstream readiness and downstream criticality. In generic form:

$$m_v^{(t+1)} = \sum_{u \in \mathcal{N}(v)} M_t\Big(h_v^{(t)}, h_u^{(t)}, \xi_{uv}\Big), \tag{14}$$

$$h_v^{(t+1)} = U_t\Big(h_v^{(t)}, m_v^{(t+1)}\Big), \tag{15}$$

followed by a readout that yields node embeddings for the policy head. This makes the policy size-invariant: the same parameters score every node and can be applied to unseen DAG sizes.

The actor outputs a logit z_i for each task t_i, forming a softmax policy:

$$\pi_\theta(a = t_i \mid s) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \tag{16}$$

where $T > 0$ is a temperature controlling exploration. The critic outputs a scalar value estimate $V_\phi(s)$.

3.5.3 Training Objective

We use an episodic actor–critic objective with terminal reward, consistent with recent learning-based scheduling formulations:[3, 7, 8]

$$A = r - V_\phi(s_0), \tag{17}$$

$$\begin{aligned}\mathcal{L}(\theta, \phi) = -A \sum_{k=0}^{R-1} \log \pi_\theta(a_k \mid s_k) \\ + \frac{1}{2}\left(V_\phi(s_0) - r\right)^2 - \lambda H(\pi_\theta).\end{aligned} \tag{18}$$

where $H(\pi_\theta)$ is policy entropy and $\lambda \geq 0$ encourages exploration.

Algorithm 4 GNN-RL Incremental Allocation (Actor–Critic)

Input: DAG G, budget E_{tot}, episodes H, batch size B
Output: Best allocation $\mathbf{e}^{\text{best}}$

```
Initialize actor parameters θ and critic parameters φ
Choose feasible base allocation e^(0); R ← E_tot − Σ_i e_i^(0)
e^best ← NULL; C^best ← +∞
for ep = 1 to H do
    for b = 1 to B do
        e ← e^(0)
        for k = 0 to R − 1 do
            Build features x from (G, e) and run MPNN to obtain logits {z_i}
            Sample a_k ~ π_θ(· | s_k) using (16)
            e_{a_k} ← e_{a_k} + 1
        end for
        C ← SimulateMakespan(G, e); r ← −C
        Update (θ, φ) by minimizing (17) on the trajectory
        if C < C^best then
            e^best ← e; C^best ← C
        end if
    end for
end for
return e^best
```

4 Experimental Design and Results

4.1 Synthetic DAG Test Set

To systematically study scalability and resource effects, we construct a synthetic workflow benchmark by randomly generating directed acyclic graphs (DAGs) with controlled size and average executor availability. Each instance is identified by a tuple (N, E) where N is the number of nodes and E is the average number of executors per node. We sweep $N \in \{100, 200, \ldots, 1000\}$ and $E \in \{2, 4, 6, 8\}$, and generate 4 independent random instances for each (N, E) configuration, resulting in 160 DAG instances in total.

Each DAG is generated by first sampling a random topological order and then adding candidate edges only from earlier to later nodes (edge probability 0.3, with basic connectivity checks), which guarantees acyclicity. Source work counts are sampled from $U\{20, 30\}$ and per-item processing times from $U\{1, 8\}$; non-source work is produced by the propagation rule in Section 2. The executor budget is $E_{\text{tot}} = NE$, so E controls the average executor availability. This design separates graph size, resource regime, and structural randomness.

We evaluate four scheduling methods: Upward-Rank, DBL, ACO, and GNN. For each instance, we report the resulting *makespan* (total completion time). In addition, for ACO and GNN we record wall-clock runtime in seconds. Generalization is tested by evaluating policies on unseen random DAG seeds across all (N, E) configurations rather than on the same instances used during policy optimization.

4.2 Visualization Metrics and Plot Construction

Because random instances may contain outliers, most plots aggregate results using the *median* across the 4 seeds per (N, E) setting.

Winner map (Fig. 1). The winner map visualizes which method achieves the lowest *median* makespan under each (N, E) configuration. Let $\mathcal{D}_{N,E}$ denote the set of instances with the same (N, E). For method m, define the aggregated makespan:

$$\tilde{M}_m(N, E) \;=\; \text{median}_{i \in \mathcal{D}_{N,E}}\; M_{m,i}, \tag{19}$$

where $M_{m,i}$ is the makespan of method m on instance i. The winner at (N, E) is then

$$w(N, E) \;=\; \arg\min_{m}\; \tilde{M}_m(N, E). \tag{20}$$

Each cell in Fig. 1 is colored by $w(N, E)$, providing a compact overview of which method dominates under different sizes and resource regimes.

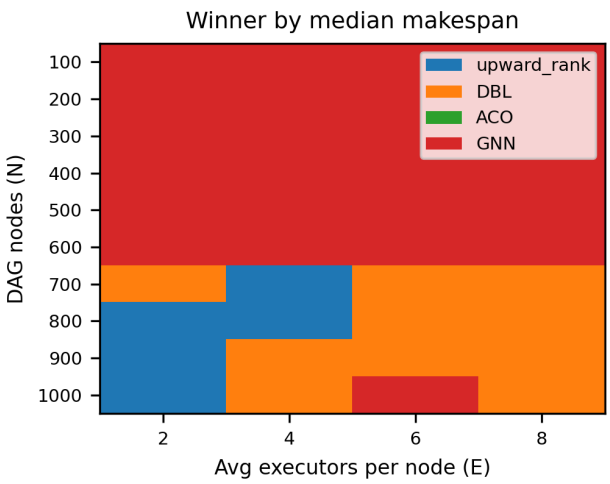

Fig. 1: Winner map over (N, E) using the median makespan per configuration. Each cell is colored by the method $w(N, E)$ that achieves the lowest median makespan among the four random seeds.

Normalized makespan distribution (Fig. 2). To compare solution quality across heterogeneous instances, we normalize makespans by the best method on each instance:

$$r_{m,i} \;=\; \frac{M_{m,i}}{\min_k M_{k,i}}, \quad r_{m,i} \geq 1. \tag{21}$$

Fig. 2 summarizes the distribution of $r_{m,i}$ (boxplot) for all methods, showing both typical performance (median) and robustness (spread). The inset shows the runtime ratio for methods with recorded time:

$$\rho_i \;=\; \frac{T_{\text{ACO},i}}{T_{\text{GNN},i}}, \tag{22}$$

where T is runtime in seconds.

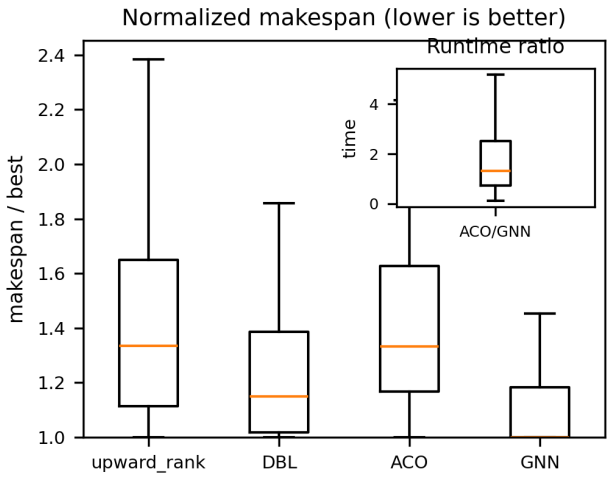

Fig. 2: Distribution of normalized makespan $r_{m,i}$ (Eq. (21)) across all instances. Lower is better and $r = 1$ denotes instance-wise optimal. The inset reports the runtime ratio $\rho_i = T_{\text{ACO}}/T_{\text{GNN}}$.

Median speedup vs. DAG size (Fig. 3). To examine how performance changes with graph size, we plot the *median* makespan ratio between each baseline b and GNN grouped by N:

$$S_{b \to \text{GNN}}(N) \;=\; \text{median}_{i:\, N_i = N} \left(\frac{M_{b,i}}{M_{\text{GNN},i}} \right). \tag{23}$$

Values above 1 indicate that GNN yields a shorter makespan than baseline b on median; values below 1 indicate the opposite.

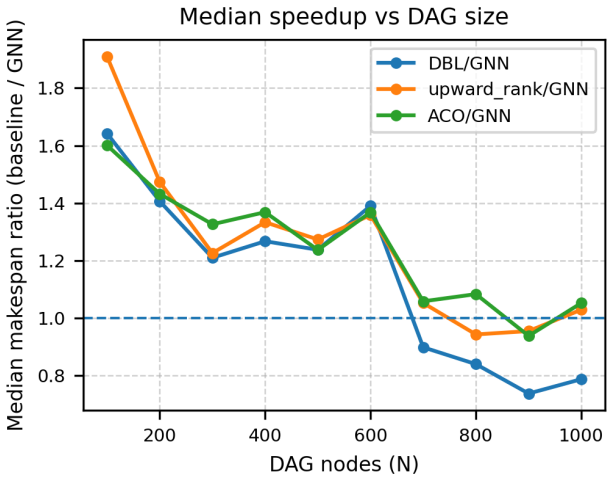

Fig. 3: Median speedup versus DAG size (Eq. (23)). Values above 1 indicate that GNN reduces makespan compared with the baseline on median; values below 1 indicate the baseline is better.

Quality–runtime trade-off (Fig. 4). Finally, we visualize the trade-off between runtime and solution quality for ACO and GNN using a scatter plot of $(T_{m,i}, r_{m,i})$ with $r_{m,i}$ defined in Eq. (21). Points closer to the lower-left corner cor-

respond to methods that are both faster and closer to the best makespan.

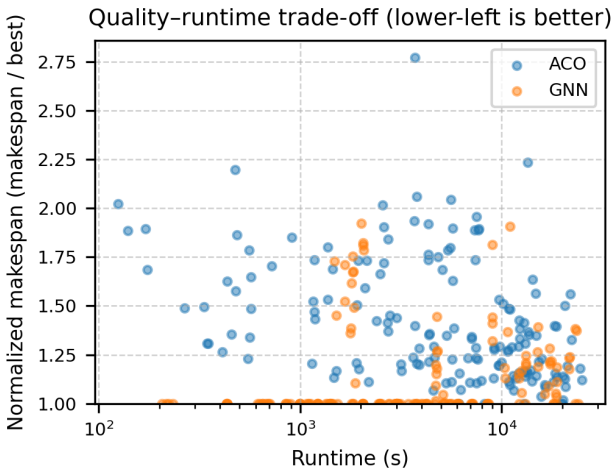

Fig. 4: Quality–runtime trade-off for ACO and GNN. Each point corresponds to one instance with coordinates (runtime in seconds, normalized makespan). Lower-left is better (faster and closer to optimal).

4.3 Results and Observed Trends

Overall, GNN achieves the best makespan on the largest fraction of instances, indicating strong average solution quality on the synthetic benchmark. Pairwise comparisons show that GNN outperforms Upward-Rank and ACO on the majority of instances, and also wins against DBL more often than not. The normalized makespan distribution further confirms that GNN is frequently near-optimal (median $r_{m,i}$ close to 1), while traditional heuristics exhibit larger dispersion, reflecting higher instance sensitivity. The advantage mainly comes from using graph embeddings to combine local workload, current executor contention, and downstream fan-out/criticality, whereas fixed heuristics emphasize only one or two of these signals.

Fig. 1 highlights that the relative strengths of methods depend on both DAG size and executor regime. In many (N, E) settings, GNN dominates, but DBL remains competitive and can become preferable in certain larger-size configurations, which is consistent with the speedup curves in Fig. 3 where the baseline/GNN median ratio may approach 1 (or dip below 1) at the high-N end. This suggests that while learning-based scheduling generalizes well across diverse random structures, strong heuristics can still match or exceed it when their bottleneck proxy happens to fit the graph.

From an efficiency perspective, Fig. 4 and the runtime inset in Fig. 2 show that GNN typically provides a favorable quality–runtime trade-off relative to ACO, with substantial variance across instances. In practice, this means GNN often delivers near-optimal makespans with lower (or comparable) time cost, whereas ACO can be competitive on some instances but may incur higher computational overhead on others. Taken together, these results indicate that GNN is a strong default choice for synthetic DAG scheduling across broad regimes, while DBL can be a valuable fallback or complementary method when scaling to the largest graphs. These cross-size and cross-executor comparisons provide the main evidence of generalization.

5 Conclusion

We formulated a pipeline DAG scheduling problem with dynamic work propagation and homogeneous, non-migratable executors, and targeted makespan minimization under resource contention. To address the coupled, time-varying nature of work availability, we developed a pipeline-adapted DBL heuristic and a GNN+RL policy that exploits DAG structure for decision making. Experiments on randomly generated DAGs across multiple sizes and executor regimes indicate consistent makespan reductions, and the learned scheduler offers a strong quality–runtime trade-off relative to ACO-style search. Future work includes evaluating on real workflow traces, extending to heterogeneous resources and communication costs, and incorporating additional operational constraints (e.g., priorities, deadlines, or cost budgets).

References

[1] Coleman, T., Casanova, H., Pottier, L., Kaushik, M., Deelman, E., & Ferreira da Silva, R. (2022). WfCommons: A framework for enabling scientific workflow research and development. *Future Generation Computer Systems, 128*, 16–27. https://doi.org/10.1016/j.future.2021.09.043

[2] NoorianTalouki, R., Hosseini Shirvani, M., & Motameni, H. (2022). A heuristic-based task scheduling algorithm for scientific workflows in heterogeneous cloud computing platforms. *Journal of King Saud University: Computer and Information Sciences, 34*(8), 4902–4913. https://doi.org/10.1016/j.jksuci.2021.05.011

[3] Lee, H., Cho, S., Jang, Y., Lee, J., & Woo, H. (2021). A global DAG task scheduler using deep reinforcement learning and graph convolution network. *IEEE Access, 9*, 158548–158561. https://doi.org/10.1109/ACCESS.2021.3130407

[4] Liu, P., Liu, B., Zhou, N., Peng, X., & Lin, W. (2022). Survey of hybrid cloud workflow scheduling. *Computer Science, 49*(5), 235–243. https://doi.org/10.11896/jsjkx.210300303

[5] Mikram, H., El Kafhali, S., & Saadi, Y. (2024). HEPGA: A new effective hybrid algorithm for scientific workflow scheduling in cloud computing environment. *Simulation Modelling Practice and Theory, 130*, 102864. https://doi.org/10.1016/j.simpat.2023.102864

[6] Shobeiri, P., Akbarian Rastaghi, M., Abrishami, S., & Shobiri, B. (2024). PCP–ACO: A hybrid deadline-constrained workflow scheduling algorithm for cloud environment. *The Journal of Supercomputing, 80*(6), 7750–7780. https://doi.org/10.1007/s11227-023-05753-8

[7] Pan, J., & Wei, Y. (2024). A deep reinforcement learning-based scheduling framework for real-time workflows in the cloud environment. *Expert Systems with Applications, 255*, 124845. https://doi.org/10.1016/j.eswa.2024.124845

[8] Liu, Z., Huang, L., Gao, Z., Luo, M., Hosseinalipour, S., & Dai, H. (2024). GA-DRL: Graph neural network-augmented deep reinforcement learning for DAG task scheduling over dynamic vehicular clouds. *IEEE Transactions on Network and Service Management, 21*(4), 4226–4242. https://doi.org/10.1109/TNSM.2024.3387707